

Search over Trajectories

Lecture 20 · November 12, 2026

Carnegie Mellon University
Language Technologies Institute

Daniel Fried

11-768: AI Agents

Repeated sampling

Generate independent trajectories in parallel from the same starting page.



Page thumbnails: Fried · VWA search talk (2024) · parallel sampling illustration ↗

Sampling methods

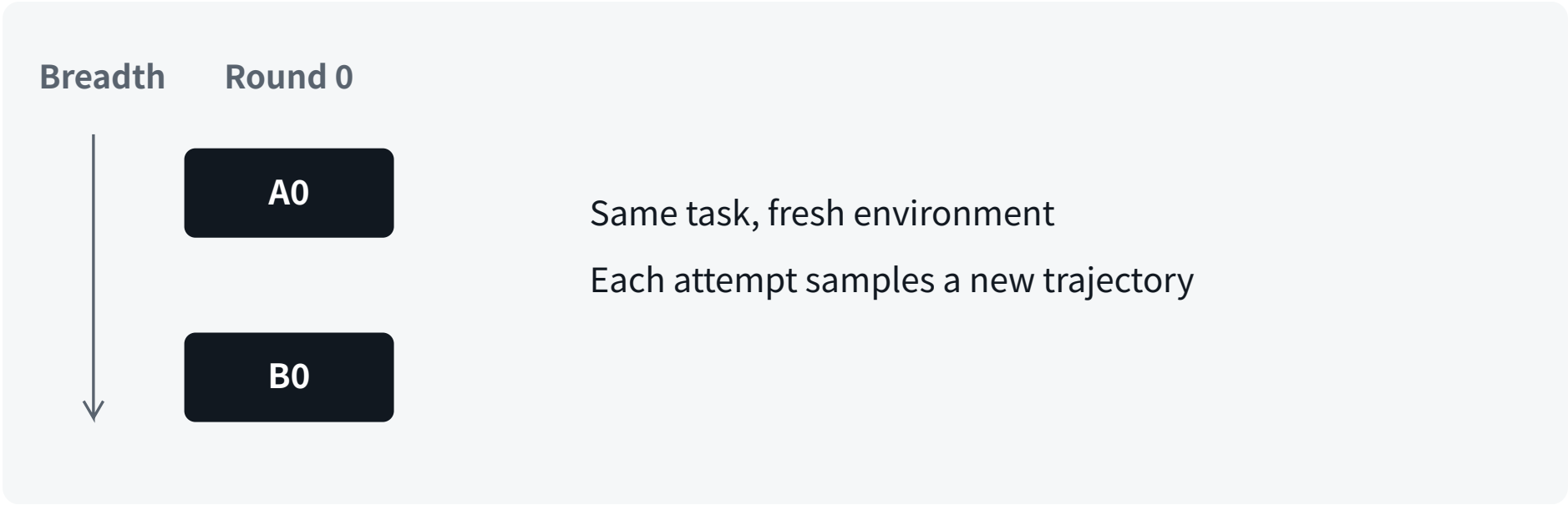
$$P_T(x_t \mid x_{<t}) = \mathrm{softmax}(z_t / T)$$

- Sample the next token from the model's distribution
- Randomness affects reasoning and tool actions
- Reasoning models may fix the temperature at **1.0**

Model-specific sampling settings · Claude docs ↗ DeepSeek-R1 usage recommendations ↗

Parallel attempts

Each attempt starts from the same task and samples independently.



- Are independent -> can often be generated in a single batch
- Each trajectory can produce an output, e.g. a patch

Coverage and pass@k

Pass@k: at least one of k attempts succeeds.

How likely is a success?

Assuming probability of success *p*:

$$P(\text{all } k \text{ fail}) = (1-p)^k$$

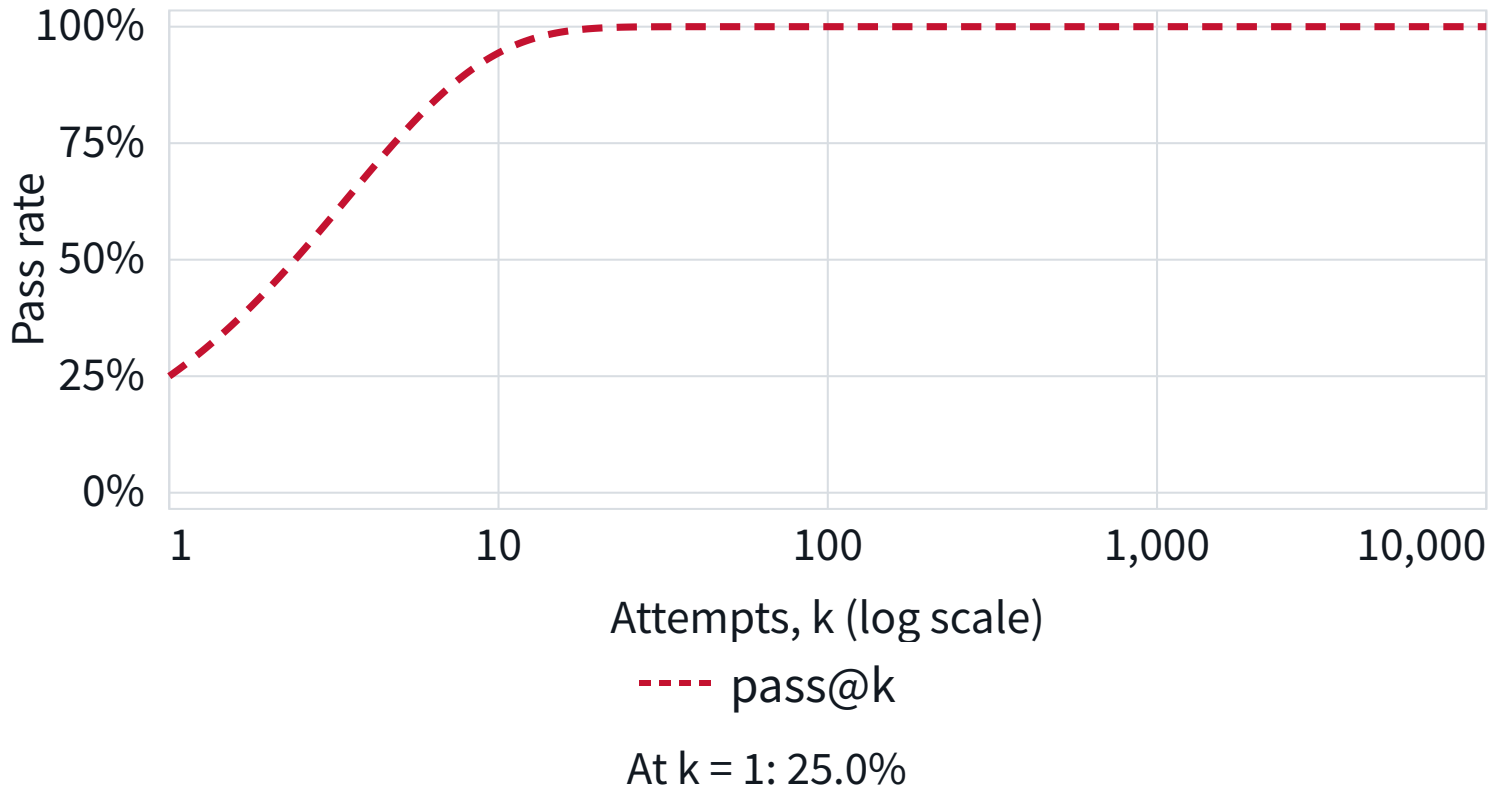
$$P(\text{at least one succeeds}) = 1 - (1-p)^k$$

Idealized coverage

Pass@k as a function of p and k:

Correct answer, p 25.0%

pass@k is the success rate you would get with an *oracle* which can perfectly distinguish correct/incorrect candidates



Independent attempts at one problem, with a fixed success probability p.

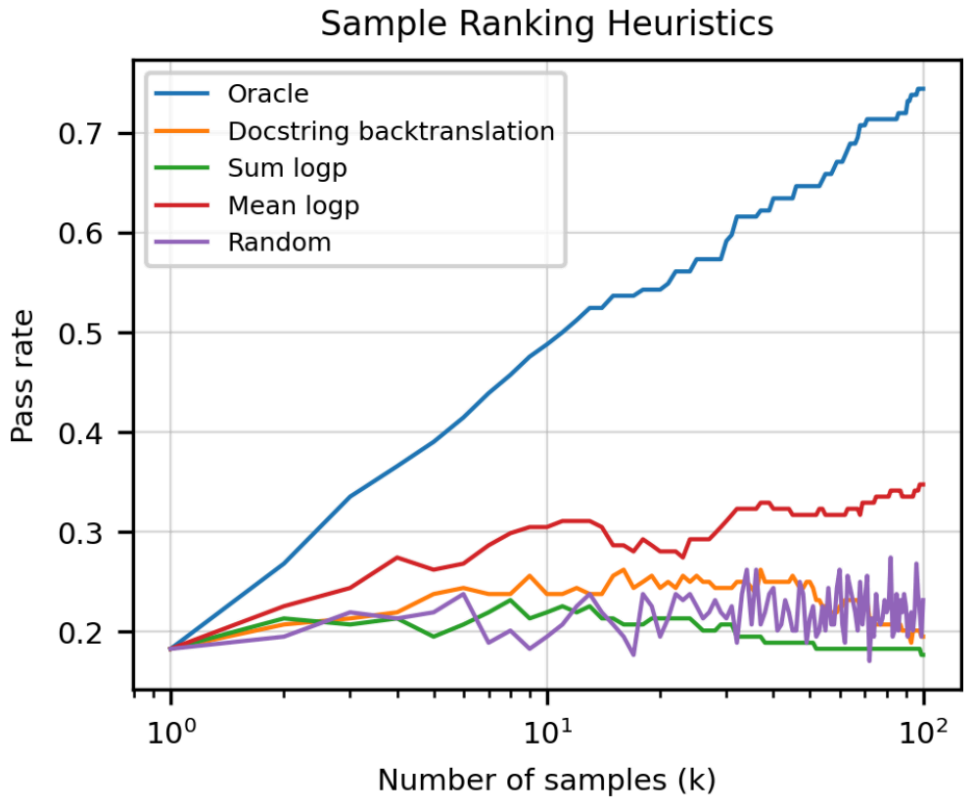
Improving coverage

- Sample more candidates!
- Increase the sampling temperature (less applicable for reasoning models)
  [Codex]
- Generate multiple responses in a single pass (less applicable for agents)
  [Verbalized Sampling]
- Use a diverse set of prompts
  [Mode-conditioning]
- Incentivize diversity / optimize for pass@k in training
  [Rewarding the Unlikely; Differential Smoothing; MaxRL]

Chen et al. (2021) ↗ Wu et al. (2025) ↗ Zhang et al. (2025/2026) ↗ He et al. (2025) ↗ Gai et al. (2026) ↗ Tajwar et al. (2026) ↗

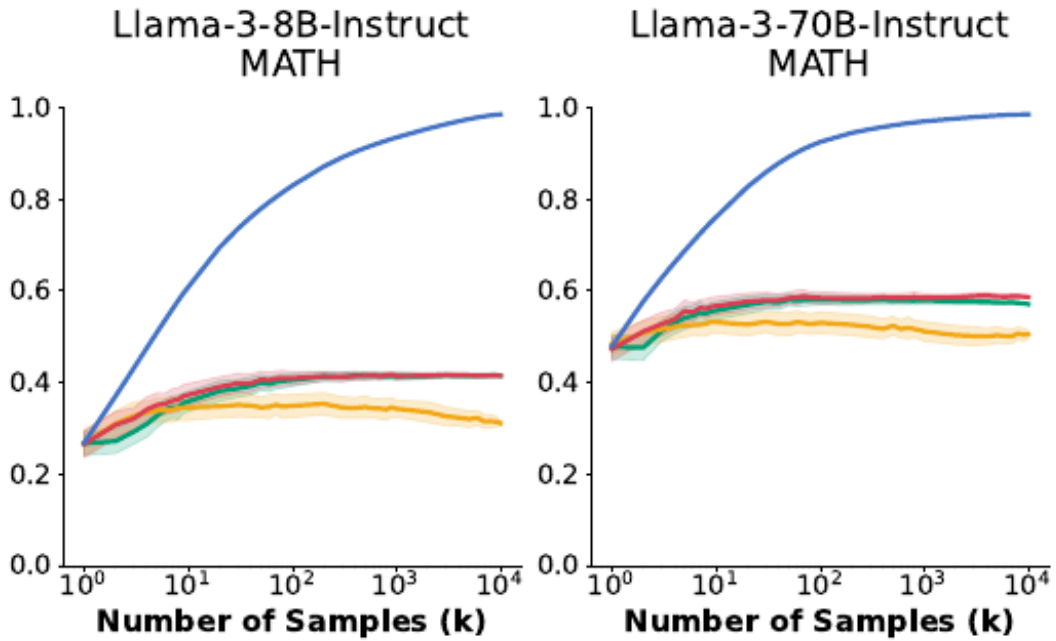
But choosing a candidate can be hard

Codex · HumanEval



A correct answer may be present

Large Language Monkeys · MATH



Coverage Output vote Best-of-N Score + vote

Practical selection can still miss it

Chen et al. · Codex (2021), Figure 7 ↗ Brown et al. · Large Language Monkeys (2024), Figure 7 ↗

Selecting an answer

Verification signals

Ground-truth checks

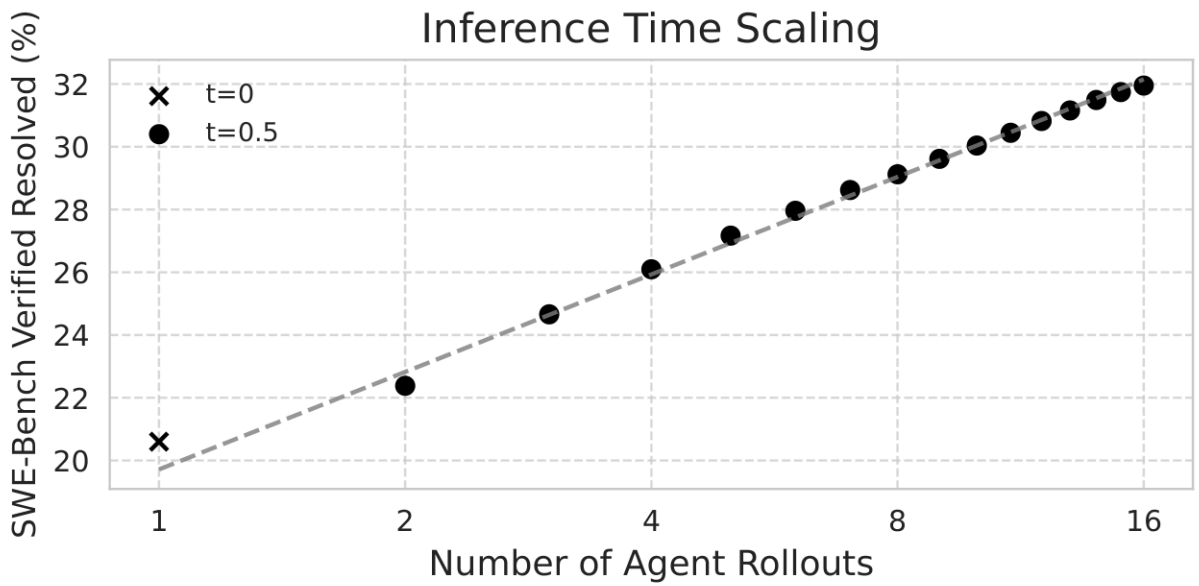
- For math: a proof checker
- For code: passes reference test cases

Predicted correctness

- For code: passes predicted test cases
- Use a reward model (e.g. LLM-as-a-judge)
- Similarity to other sampled outputs

Madaan et al. · Self-Refine (2023), § 2 ↗

Use a reward model

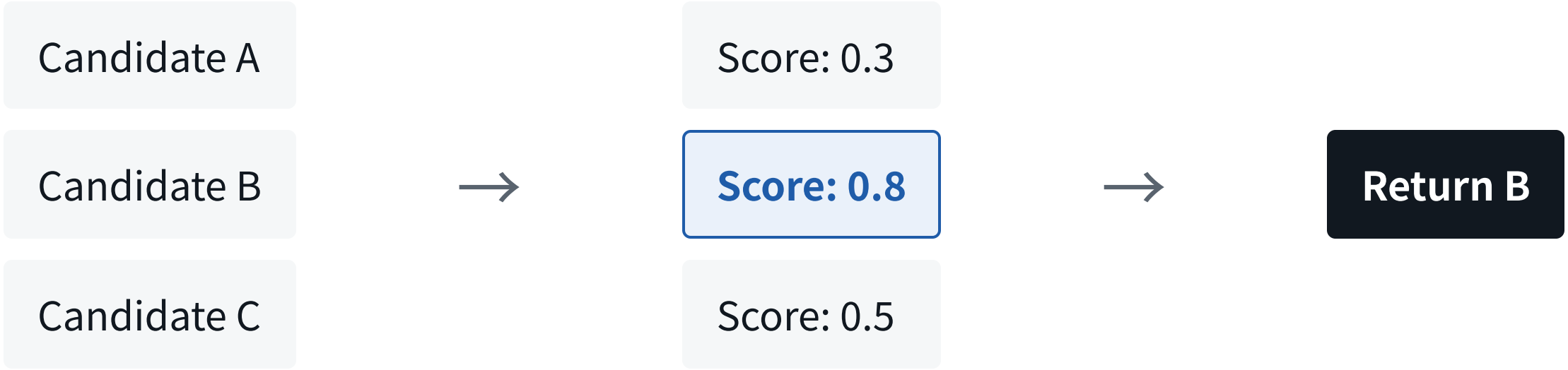


SWE-Gym trains an outcome verifier on complete OpenHands runs.

- Use task tests to label each training run
- Balance YES / NO examples; fine-tune on complete runs
- Score new runs by YES probability; return the highest

Pan et al. · SWE-Gym (2024/2025), § 5.1.1 & Appendix B.6 · training schematic; Figure 1 (bottom) ↗

Best-of-N



- Score each candidate with a reward model
- Return the candidate with the highest score

Self-consistency

Different reasoning paths can lead to the same final answer.

Reasoning A	Reasoning B	Reasoning C
42	**17**	**42**

Minimum Bayes risk with 0–1 answer loss

$$\hat{a}_{\mathrm{MBR}} = \arg\max_a \underbrace{\sum_z p(z, a \mid x)}_{p(a|x)} \approx \arg\max_a \sum_{i=1}^{N} \mathbf{1}\{a^{(i)} = a\}$$

Sum over trajectories z to get answer probabilities; approximate with sample votes.

Wang et al. · Self-Consistency (2022/2023) ↗ Welleck et al. · From Decoding to Meta-Generation (2024), § 4.2.2 ↗

Consensus over execution results

Task: sum the even numbers up to and including `n = 12`.

Candidate A

```
evens = range(2, n + 1, 2)
return sum(evens)
```

↓ Execute

42

Candidate B

```
evens = range(2, n, 2)
return sum(evens)
```

↓ Execute

30

Candidate C

```
m = n // 2
return m * (m + 1)
```

↓ Execute

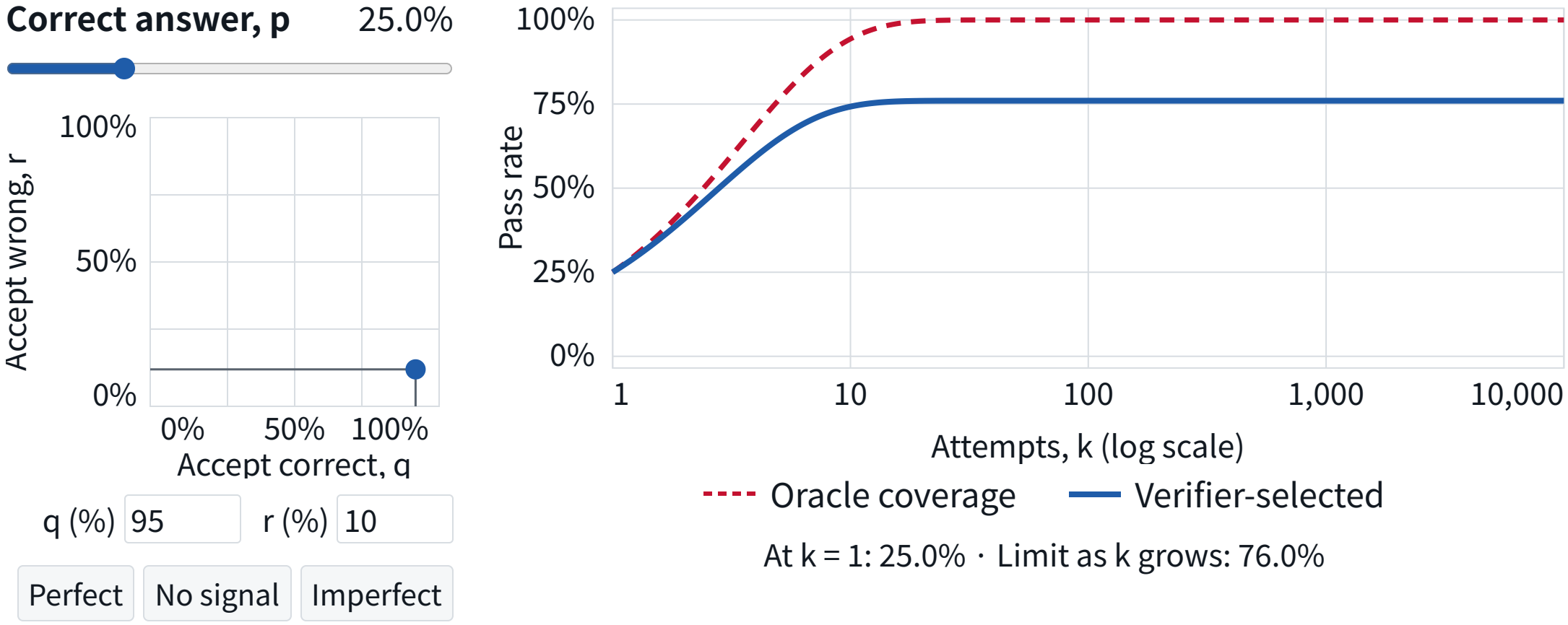
42

Execute programs and group by their outputs.

Shi et al. · MBR-Exec (2022) ↗ Li et al. · AlphaCode (2022) ↗

Imperfect verification

Verifier / reward model errors limit the gains from repeated sampling.

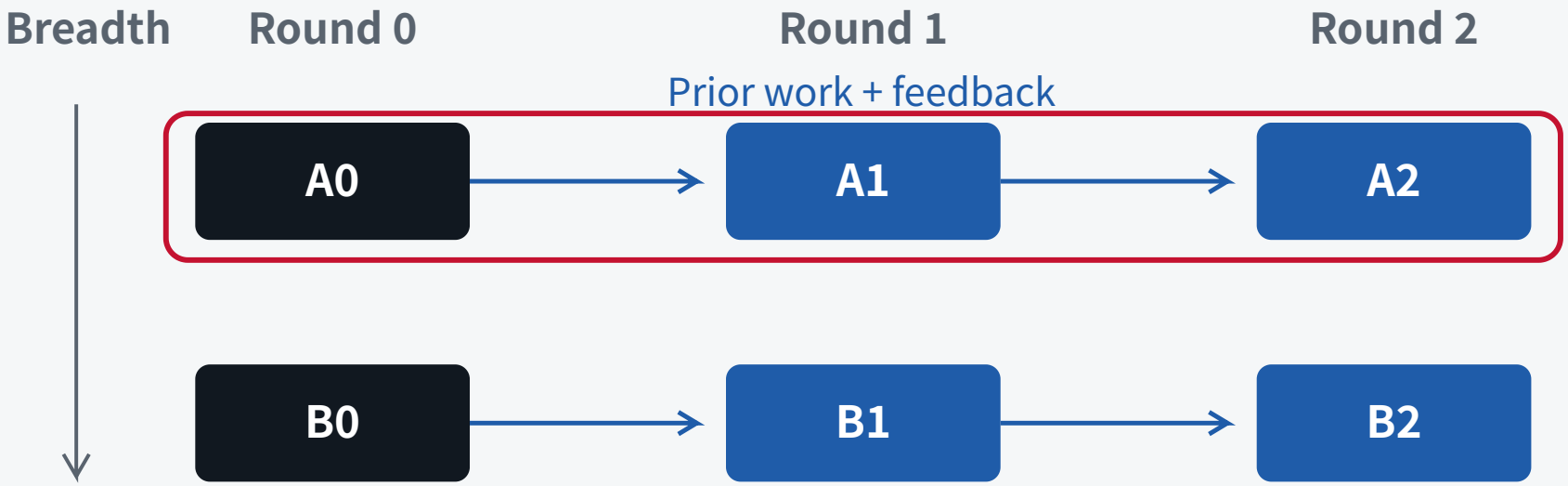


Return the first accepted answer; otherwise return the last response.

Iterative refinement

Refinement

Refinement uses feedback to revise candidate solutions. Must be done sequentially, but can refine multiple candidates in parallel



Remembering feedback on trajectories

attempt → **evaluator: compute or predict reward** → **reflection: generate feedback** → retry the same task

ATTEMPT 1

“What was a series of battles … fought on October 28, 1776 near White Plains, New York?”

`Finish[Battle of White Plains]` **incorrect**

GENERATED FEEDBACK

“… the question asked for a series of battles, but I only provided the name of one battle … I will make sure to provide more context, such as the name of the campaign …”

ATTEMPT 2

`Finish[The New York and New Jersey campaign]` **correct**

Reflexion · Shinn et al., NeurIPS 2023 ↗

Types of feedback

Basically parallel the verification signals we saw earlier:

Ground-truth checks

- For math: proof-checker errors
- For code: reference test results

  [Self-Debugging]

Predicted correctness

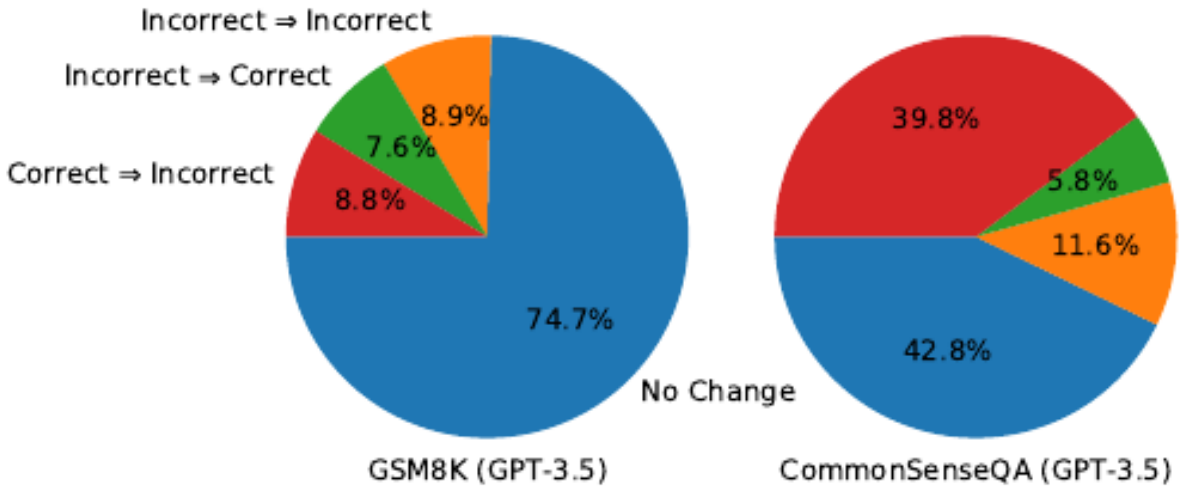
- For code: predicted test results

  [Reflexion]
- Use a reward model (e.g. LLM-as-a-judge)

  Can be the agent itself [Self-Refine]
- Similarity to other candidates [PDR]

Chen et al. · Self-Debugging (2023), § 3 ↗ Shinn et al. · Reflexion (2023), § 4.3 ↗ Madaan et al. · Self-Refine (2023), § 2 ↗

Limits of refinement



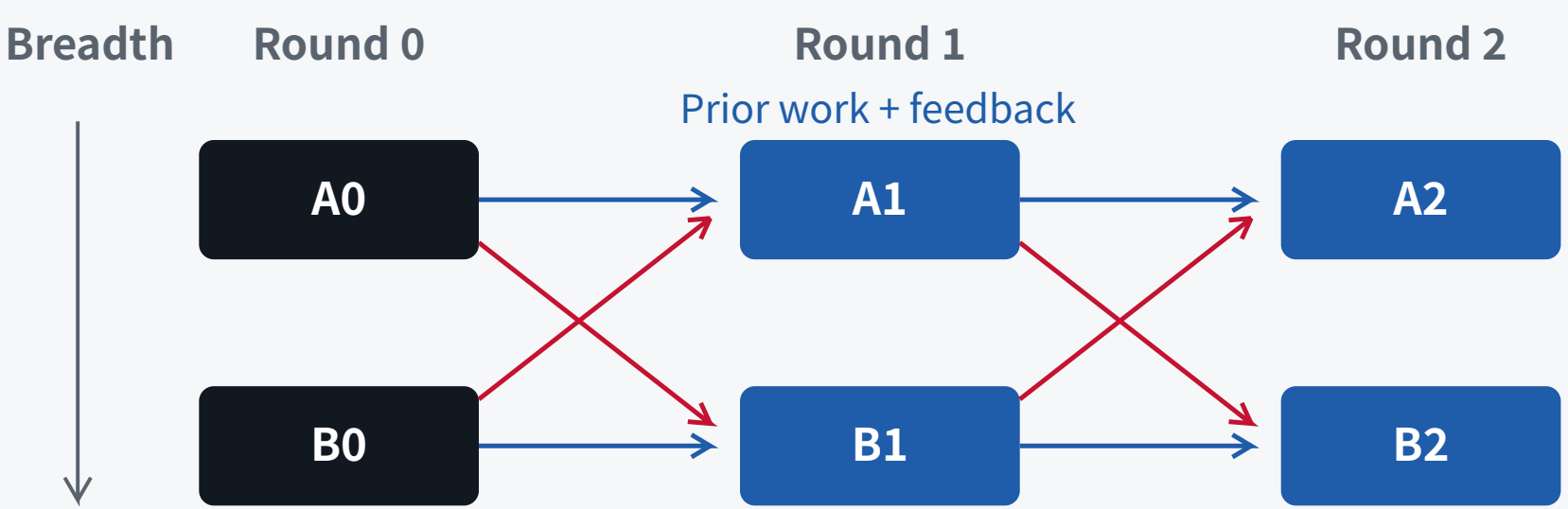
Without oracle correctness labels:

- Some incorrect answers become correct
- Some correct answers become incorrect
- Self-critique can preserve the original blind spot

Huang et al. · Intrinsic self-correction study (2023/2024), Figure 1 ↗

Combining exploration and refinement

Parallel-Distill-Refine: generate a summary of multiple candidates ("distill"); condition on this summary in later attempts.



Madaan et al. · PDR (2025), Figure 2 · shared-summary variant ↗

Related: Recursive Self-Aggregation ↗

Putting it together for agents

Summarization

Agents trajectories are long. Use an LLM to generate a structured summary of each trajectory:

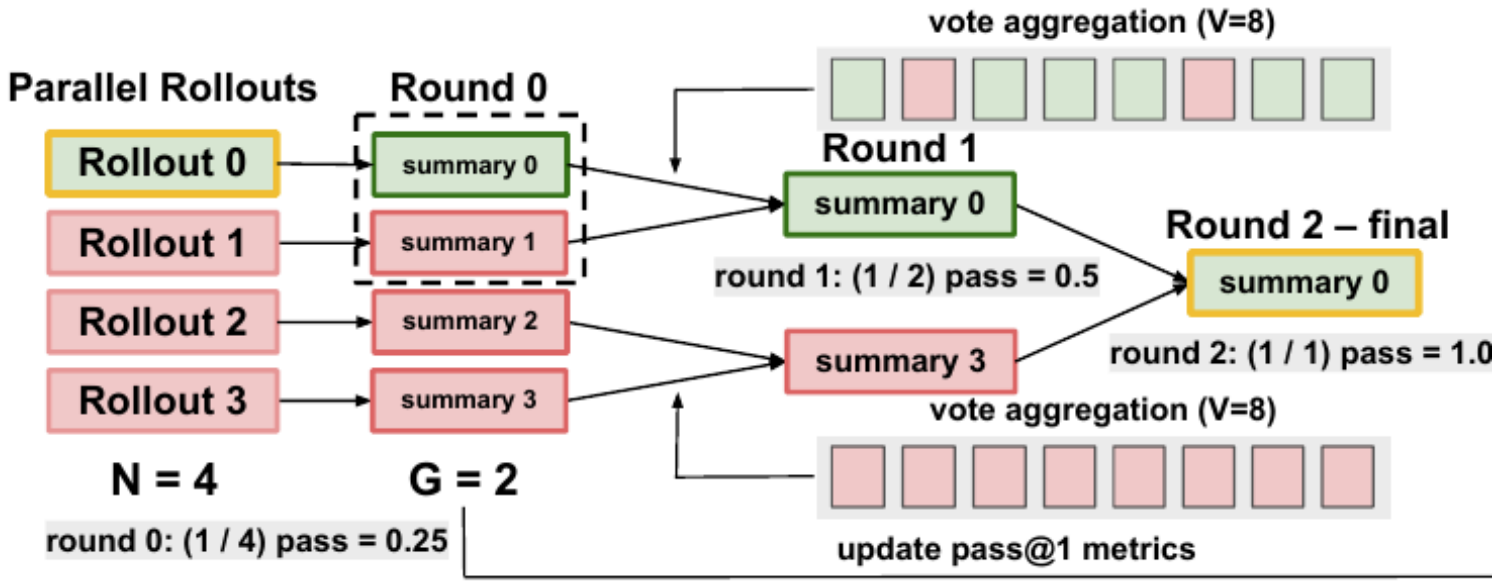
```
"key_discoveries": [
  "Add(ZeroMatrix, ZeroMatrix) evaluates to scalar 0",
  "MatAdd(0, ZeroMatrix, ZeroMatrix) canonicalizes to 0 because rm_id removes all
identities and keeps the first one (0)"
]
```

```
"solution_approach": {
  "strategy": "Modify `get_postprocessor` in `sympy/matrices/expressions/matexpr.py`
to check if `nonmatrices` is empty. […]"
}
```

Kim et al. · Agentic Coding (2026), Appendix G, Figure 23 (excerpts) ↗

Recursive selection

Recursive Tournament Voting compares rollout summaries in small groups.



- Have an LLM-as-a-judge select from multiple summaries
- The judge may be noisy; aggregate multiple decisions
- Advance the winning summary / rollout; repeat until one rollout remains

Kim et al. · Agentic Coding (2026), § 2.3 & Figure 2 ↗

Selection and reuse

As in PDR, have rollouts condition on selected summaries. Also use selection at the end.

100 SWE-bench tasks; mean rollout success before final selection.

Model	Single-rollout		Random-K		Select-K	
	Iter 0	Iter 1	Iter 0	Iter 1	Iter 0	Iter 1
Claude-4.5-Sonnet	69.87	70.87	69.87	75.06	69.87	**78.06**
Gemini-3.1-Pro	72.69	73.75	72.69	76.94	72.69	**79.25**

Multiple summaries help; using RTV to select them helps further in this ablation.

Kim et al. · Agentic Coding (2026), Figure 3 & Table 1 ↗

Harness optimization

Agent harnesses

Recall: a **harness** is the code that controls how we use a model.

Part of a harness for text classification via few shot examples.

```
examples_section = self._format_examples_section(seed=seed)
prompt = PROMPT_TEMPLATE.format(
    examples_section=examples_section,
    input=input,
)
response = self.call_llm(prompt)
answer = extract_json_field(response, "final_answer")
```

Editable harness code

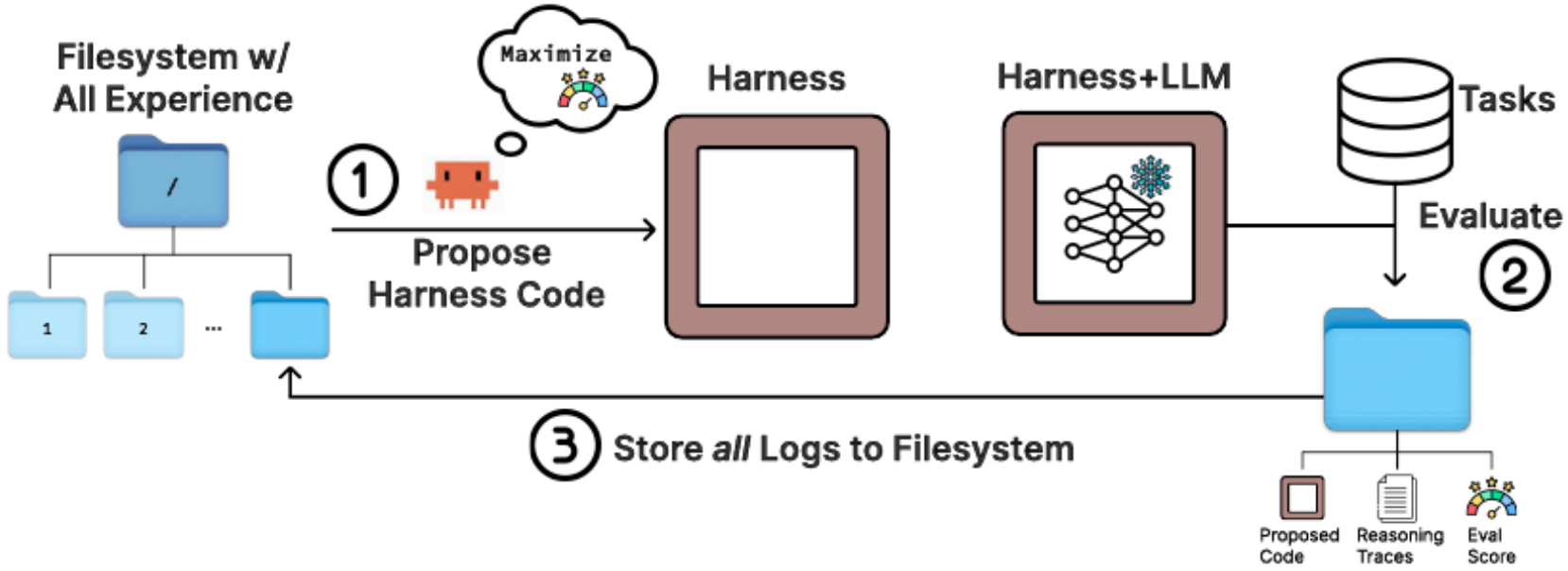
Fixed model weights

Lee et al. · Meta-Harness (2026), § 3 · released text-classification baseline ↗

Harness search

Given fixed model weights, optimize the harness around it to maximize task reward.

Store all past experiences in a file system. Use a coding agent to edit the harness.



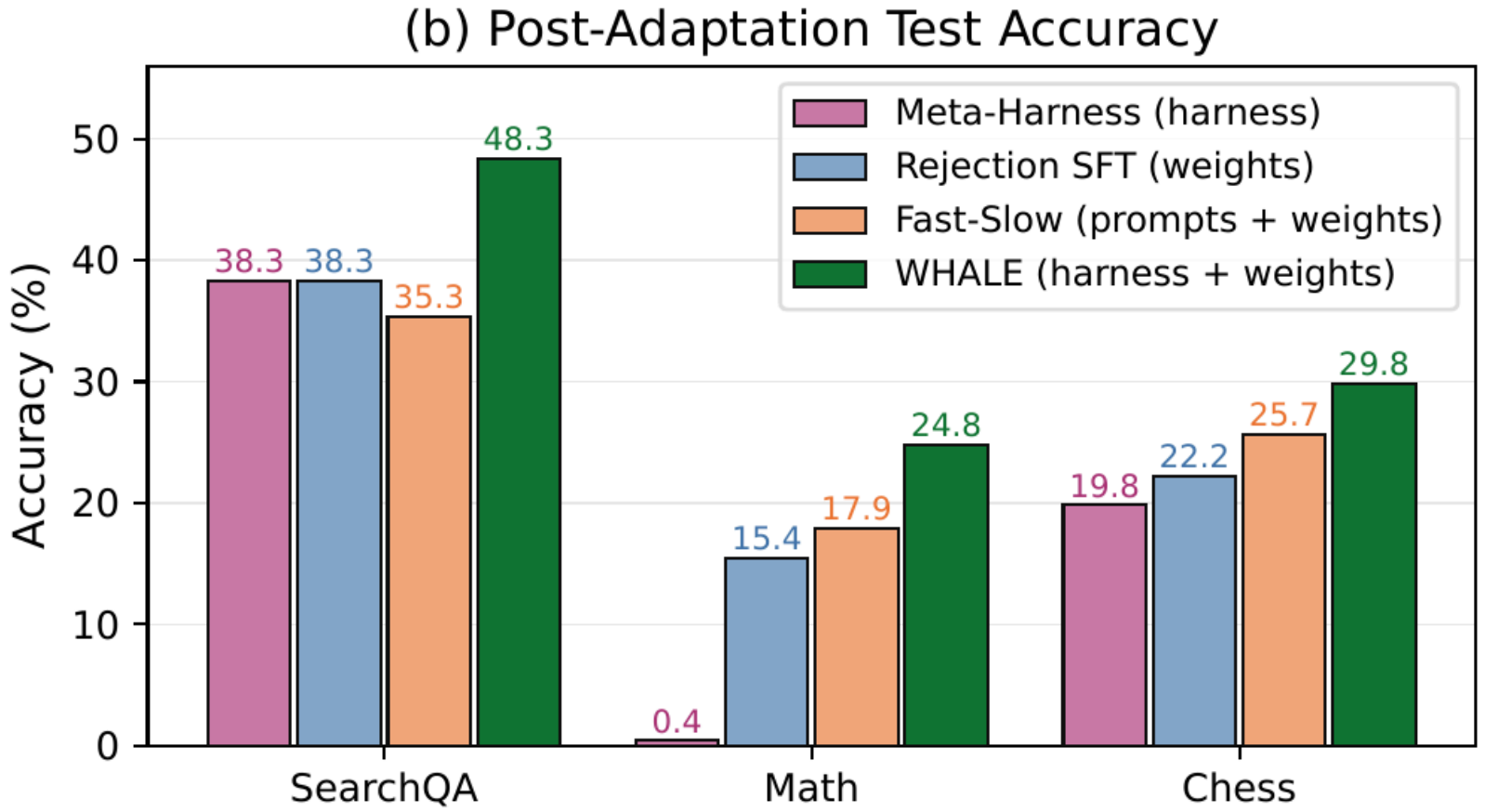
Proposer agent: reads prior code, scores, and traces; edits the harness.

Task evaluation: run the current harness with the fixed LLM weights; store logs and code.

Lee et al. · Meta-Harness (2026), Figure 2 ↗

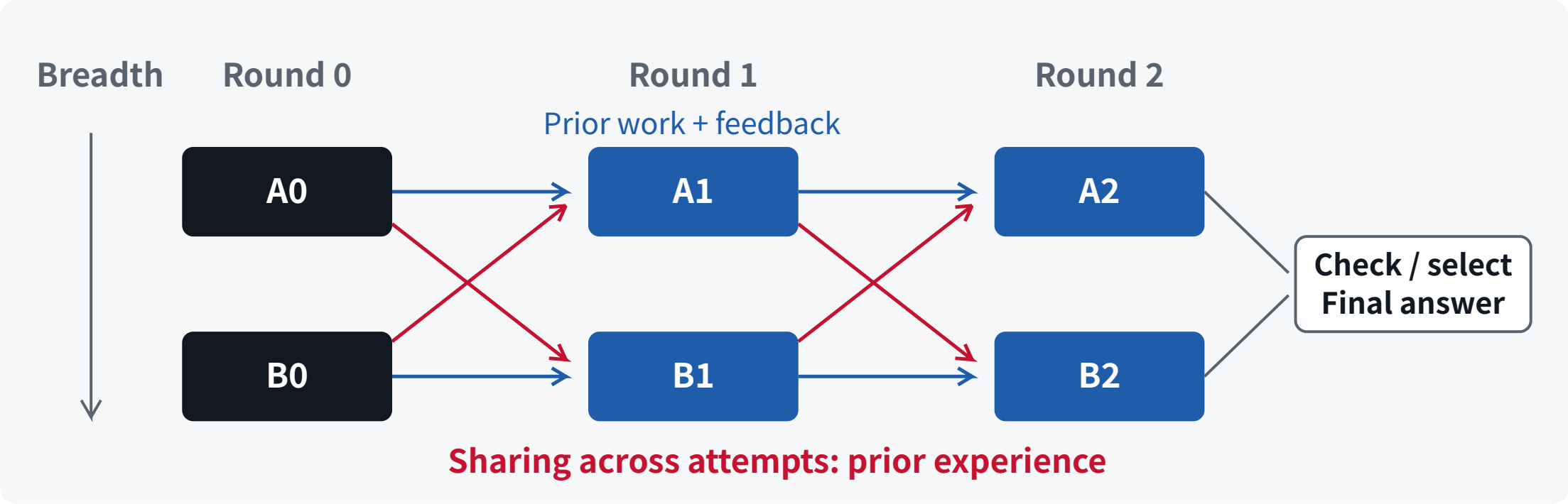
Joint weight and harness optimization

WHALE: joint model training and harness search via alternating optimization



Kim et al. · WHALE (2026), Figure 1(a–b), § § 3–4 ↗

Where would you spend more compute?



- If pass@k scores are low: increase breadth or feedback conditioning
- If final accuracy is much lower than pass@k: improve the selector



Thank you, Questions?

Next class: Advanced search